

La casa di pietra

Arben ha ereditato un bellissimo appezzamento di terreno sulle colline che dominano Argirocastro. Ispirato dalle famose case dai tetti di pietra della città, ha intenzione di costruire una casa tradizionale tutta sua.

Il terreno è rappresentato come una griglia quadrata $N \times N$, in cui le righe e le colonne sono numerate da 1 a N . Il terreno roccioso ha una caratteristica particolare: c'è esattamente un grande masso inamovibile in ogni riga e in ogni colonna della griglia.

Per gettare le fondamenta della sua casa, Arben deve selezionare una **sottogriglia quadrata vuota** — una regione di dimensione $S \times S$ che non contiene alcun masso.

Sia M il massimo intero tale che, per *qualsiasi* disposizione dei massi che soddisfi le condizioni sopra descritte, sia garantita l'esistenza di una sottogriglia quadrata vuota di dimensione $M \times M$. Arben vuole trovare *una qualsiasi* sottogriglia quadrata vuota di dimensione almeno $M \times M$.

Per ispezionare il terreno, Arben chiede aiuto alla sua amica Besa. Besa ha portato uno speciale strumento di misurazione chiamato **Gurëmatës** ("contatore di pietre").

In una singola operazione, Besa può configurare il Gurëmatës per scansionare una sottogriglia rettangolare data dal suo angolo in alto a sinistra (x_1, y_1) e dal suo angolo in basso a destra (x_2, y_2) . Lo strumento restituisce il numero esatto di massi in quella regione. Il Gurëmatës è alimentato da una piccola batteria e ogni scansione consuma molta energia, quindi Besa deve usarlo il minor numero di volte possibile.

Scrivi un programma che usi il Gurëmatës per individuare una sottogriglia quadrata vuota di dimensione $S \times S$ con $S \geq M$, e ne comunichi la posizione.

Dettagli di implementazione

Devi implementare la seguente procedura:

```
std::vector < int > solve(int N);
```

- N : la dimensione della griglia.

- La procedura deve restituire esattamente quattro interi $\{x_1, y_1, x_2, y_2\}$: gli angoli in alto a sinistra e in basso a destra della sottogriglia quadrata vuota che ha trovato. Deve valere la condizione $x_2 - x_1 = y_2 - y_1 = S - 1$ con $S \geq M$.
- Questa procedura viene chiamata esattamente una volta per ogni caso di test.

La procedura può effettuare chiamate alla seguente procedura:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : la riga e la colonna dell'angolo in alto a sinistra della sottogriglia scansionata.
- x_2, y_2 : la riga e la colonna dell'angolo in basso a destra della sottogriglia scansionata.
- Devono valere $1 \leq x_1 \leq x_2 \leq N$ e $1 \leq y_1 \leq y_2 \leq N$. Altrimenti, il tuo programma viene terminato immediatamente e riceve 0 punti per quel caso di test.
- La procedura restituisce il numero di massi nella sottogriglia scansionata.

Il grader non è adattivo: le posizioni dei massi sono fissate prima che `solve` venga chiamata e non dipendono dalle tue chiamate ad `ask`.

Vincoli

- $2 \leq N \leq 500$
- Puoi effettuare al più $2.5 \cdot 10^5$ chiamate ad `ask`.

Subtask

Subtask	Punti	Vincoli aggiuntivi
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Nessun vincolo aggiuntivo.

Punteggio

Se `solve` restituisce una regione non valida, effettua una chiamata non valida ad `ask`, oppure chiama `ask` più di $2.5 \cdot 10^5$ volte, ricevi 0 punti per quel caso di test.

Altrimenti, sia Q il numero di chiamate ad `ask`. Per quel caso di test ricevi i punti del subtask moltiplicati per $\min(1, f(Q))$, dove

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & \text{se } Q \leq 15 \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & \text{se } Q > 15 \end{cases}$$

Il tuo punteggio per un subtask è il punteggio minimo su tutti i casi di test di quel subtask. Il punteggio finale è la somma dei punteggi dei subtask.

Per aiutarti a stimare il tuo punteggio, la tabella seguente mostra il moltiplicatore approssimativo per alcuni valori specifici di Q :

Numero di query (Q)	Moltiplicatore ($\approx$)
$1 \leq Q \leq 11$	1.00
12	0.95
15	0.80
50	0.70
100	0.64
1000	0.45
10000	0.26
100000	0.07
$Q \geq 233573$	0.00

Grader di esempio

Il grader di esempio legge l'input nel seguente formato:

```
N
c_1 c_2 ... c_N
```

Qui, c_i è la colonna del masso nella riga i ($1 \leq c_i \leq N$).

Il grader di esempio stampa una singola riga contenente il moltiplicatore $\min(1, f(Q))$ ottenuto dalle tue query, oppure 0 se la sottogriglia restituita o una qualsiasi chiamata ad `ask` non è valida.

Esempi

Esempio 1

Input del grader di esempio:

```
4
1 2 3 4
```

I massi si trovano in $(1, 1)$, $(2, 2)$, $(3, 3)$ e $(4, 4)$. Il tuo programma non lo sa.

Il grader chiama `solve(4)`. Una possibile sequenza di chiamate è:

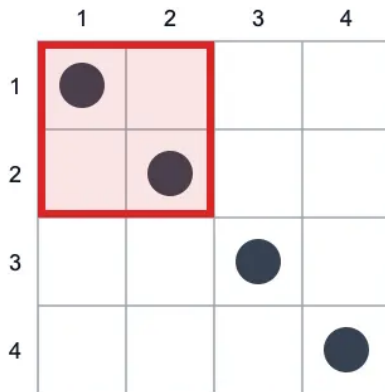
- `ask(1, 1, 2, 2)` restituisce 2.
- `ask(2, 4, 4, 4)` restituisce 1.
- `ask(3, 1, 3, 2)` restituisce 0.

Poi `solve` restituisce $\{1, 3, 2, 4\}$.

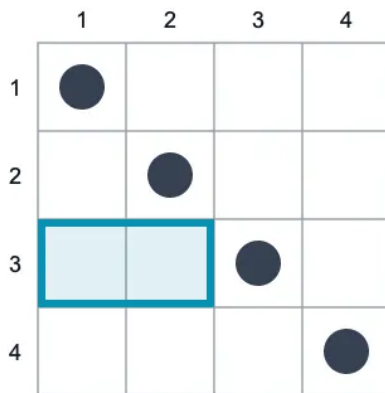
Output del grader di esempio:

```
1
```

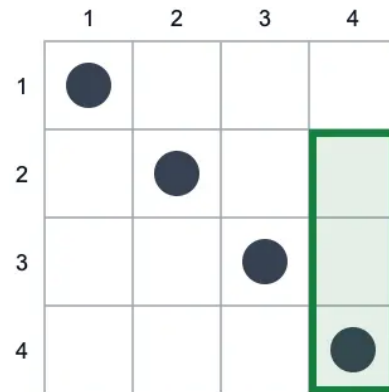
La prima chiamata scansiona il blocco 2×2 in alto a sinistra e trova i massi in $(1, 1)$ e $(2, 2)$. La seconda scansiona le righe da 2 a 4 della colonna 4 e trova il masso in $(4, 4)$. La terza scansiona le prime due celle della riga 3 e non trova alcun masso. La regione restituita copre le righe da 1 a 2 e le colonne da 3 a 4, che è una sottogriglia vuota 2×2 .



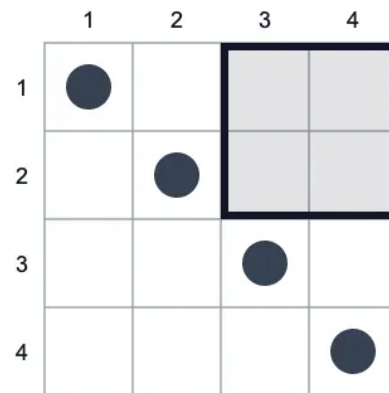
ask(1, 1, 2, 2) returns 2



ask(3, 1, 3, 2) returns 0



ask(2, 4, 4, 4) returns 1



return {1, 3, 2, 4}

Per $N = 4$, il programma ha usato $Q = 3$ chiamate, quindi $f(Q) = 1.4$ e il moltiplicatore è 1. Sono accettate anche altre risposte, per esempio $\{3, 1, 4, 2\}$, $\{1, 2, 1, 2\}$, $\{3, 2, 3, 2\}$, $\{3, 4, 3, 4\}$ o qualsiasi cella vuota, dato che $M = 1$ per $N = 4$.